This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

End-to-End Hierarchical Reinforcement Learning With Integrated Subgoal Discovery

Shubham Pateria, Budhitama Subagdja, Ah-Hwee Tan, *Senior Member, IEEE*, and Chai Quek, *Senior Member, IEEE*

***Abstract*— Hierarchical reinforcement learning (HRL) is a promising approach to perform long-horizon goal-reaching tasks by decomposing the goals into subgoals. In a holistic HRL paradigm, an agent must autonomously discover such subgoals and also learn a hierarchy of policies that uses them to reach the goals. Recently introduced end-to-end HRL methods accomplish this by using the higher-level policy in the hierarchy to directly search the useful subgoals in a continuous subgoal space. However, learning such a policy may be challenging when the subgoal space is large. We propose integrated discovery of salient subgoals (LIDOSS), an end-to-end HRL method with an integrated subgoal discovery heuristic that reduces the search space of the higher-level policy, by explicitly focusing on the subgoals that have a greater probability of occurrence on various state-transition trajectories leading to the goal. We evaluate LIDOSS on a set of continuous control tasks in the MuJoCo domain against hierarchical actor critic (HAC), a state-of-the-art end-to-end HRL method. The results show that LIDOSS attains better goal achievement rates than HAC in most of the tasks.**

***Index Terms*— Hierarchical reinforcement learning (HRL), reinforcement learning, subgoal discovery.**

I. Introduction

HIERARCHICAL reinforcement learning (HRL) is a promising approach for performing long-horizon goal-reaching tasks, in which an agent needs to execute a long sequence of actions in a large state space to reach a goal state [1]–[7]. A particular form of HRL, referred to as Feudal HRL [4], [8], addresses the long-horizon goal-reaching problem by learning a hierarchy of policies that decompose the goal into a sequence of easily reachable subgoals. A subgoal can be a state in the original state space or in an abstract state space. In such a hierarchy, a higher-level policy learns to choose a sequence of subgoals, as its outputs, to reach the goal. Meanwhile, a lower-level policy learns to choose a sequence of actions to reach a subgoal chosen by the higher-level policy.

The main challenge in Feudal HRL is to train the hierarchy of policies in an end-to-end manner. This means that neither the subgoals are predefined nor the policies are pretrained. Recent approaches achieve this by directly using the subgoal space as the output space of the higher-level policy, where the subgoal space is a continuous vector space containing all possible subgoals [2], [3]. Therefore, the agent does not require predefined subgoals and can directly learn to choose the subgoals by training the higher-level policy through reward maximization. This, however, requires the higher-level policy to incrementally update its output over a large subgoal space, which requires a long training time.

This article investigates the benefits of subsampling the output space of the higher-level policy to improve the rate at which the agent learns to reach the goals, in episodic goal-reaching tasks. In this regard, we propose a novel subgoal discovery heuristic, which estimates the probability of occurrence (or simply *occurrence probability*) of different subgoal states on various state-transition trajectories[1] leading to a goal during training, finds the subgoals with higher occurrence probabilities within local neighborhoods in the subgoal space, and selects those subgoals as the salient subgoals. The salient subgoals are then used as the outputs of the higher-level policy. The intuition behind the proposed heuristic is that a subgoal with a higher occurrence probability recorded after multiple episodes is more likely to appear on various possible trajectories leading to the goal in the future episodes as well. Hence, such a subgoal can be chosen by the higher-level policy as an intermediate target to reach the goal.

The complete method, with the subgoal discovery heuristic integrated with the end-to-end training of the hierarchy of policies, is named Hierarchical Reinforcement learning with integrated discovery of salient subgoals (LIDOSS). We evaluated LIDOSS on a set of continuous control tasks in the MuJoCo domains [9], with fixed goals and dynamic goals, whereby its performance was compared with that of hierarchical actor critic (HAC) [2], a state-of-the-art end-to-end HRL method, which uses a continuous subgoal space as the output space of the higher-level policy without subgoal discovery. Additionally, we also compared LIDOSS with other heuristics for subsampling the output space of the higher-level policy. The results showed that LIDOSS outperforms HAC and other heuristics in most of the tasks.

Manuscript received September 18, 2020; revised April 16, 2021; accepted May 29, 2021. This work was supported in part by the National Research Foundation, Singapore through its AI Singapore Program (AISG) under Award AISG2-RP-2020-019 and in part by the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier-1 under Grant 19-C220-SMU-023. *(Corresponding author: Shubham Pateria.)*

Shubham Pateria and Chai Quek are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: shubham007@e.ntu.edu.sg; ashcquek@ntu.edu.sg).

Budhitama Subagdja and Ah-Hwee Tan are with the School of Computing and Information Systems, Singapore Management University, Singapore 178902 (e-mail: budhitamas@smu.edu.sg; ahtan@smu.edu.sg).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TNNLS.2021.3087733.

Digital Object Identifier 10.1109/TNNLS.2021.3087733

[1] A state-transition trajectory is the sequence of states observed during an episode.

2162-237X © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

LIDOSS was first introduced in our previous work [10]. It is further improved in this article as follows: First, the previous implementation of LIDOSS used a deep Q network (DQN) [11], which took a state and a goal as the inputs, and predicted the Q-values of all subgoals that could be chosen by the higher-level policy. The outputs of the DQN (that is, the predicted Q-values) were independent of each other. On the contrary, the new implementation of LIDOSS presented in this article uses a single-output Q network that takes a subgoal as an input, along with a state and a goal, and predicts the Q-value of that subgoal (refer to Section IV-A). This change has been made because the single-output Q network approximates the higher-level Q-value function in the continuous subgoal space by taking a subgoal as one of the inputs; hence, it can better generalize the predicted Q-value to the previously unseen subgoals. This is important because the subgoals in LIDOSS are incrementally discovered. Such a generalization cannot be achieved using DQN, which predicts independent Q-values of different subgoals.

Second, the previous implementation of LIDOSS only discovered the salient subgoals with respect to the exogenous goal specified as part of the goal-reaching task. If the agent did not reach that goal in an episode, the episodic data were not used for subgoal discovery. The new implementation of LIDOSS uses the episodic data more prudently by treating the terminal state reached in an episode as a *proxy* goal and discovering different sets of subgoals with respect to the different proxy goals encountered (refer to Section IV-B). This is especially favorable in a task with dynamically changing goals (refer to Section V) because the subgoals discovered with respect to a proxy goal encountered in the earlier episodes can be useful if the exogenous goal in a future episode is the same as that proxy goal.

The new implementation of LIDOSS shows better performance than the previous implementation. The ablative comparison of these implementations is provided in Appendix E of the Supplementary Material.

The rest of this article is organized as follows: The relevant related work is covered in Section II. The essential preliminary concepts are discussed in Section III. Section IV describes the proposed method LIDOSS. This is followed by the discussion of experiments and results in Section V. Finally, this article is concluded in Section VI.

II. Related Work

Most modern Feudal HRL methods are based on the tabular Q-learning "manager" and "submanager" hierarchy conceptualized by Dayan and Hinton [8]. In this concept, the policy at each level in the hierarchy is regarded as the "manager," which chooses the subgoals for the lower-level policy as well as the "submanager," which achieves the subgoals given by the higher-level policy (refer to Section III-B). Vezhnevets *et al.* [4] extended this concept and proposed an end-to-end Feudal HRL method comprising a hierarchy of deep neural networks, for learning in high-dimensional state spaces. A higher-level network chooses a subgoal vector from a learned latent subgoal space. A lower-level network learns to take the actions to achieve the subgoal. This method, called feudal networks (FuN), shows better performance than a nonhierarchical agent. However, FuN requires extensive training to learn the latent subgoal space. Nachum *et al.* [3] relax this requirement by directly using the state space of the agent as the subgoal space in a method called HRL with off-policy correction (HIRO). But the more significant contribution of this work is an off-policy correction method to deal with the issue of nonstationarity in the higher-level Q-value updates encountered when multiple policies in the hierarchy are being trained simultaneously. Concurrently, Levy *et al.* [2] proposed a method called HAC, which addresses the similar nonstationarity issue, uses the concept of hindsight experience replay (HER) [12] to learn the HRL agent with sparse rewards, and also provides a subgoal testing procedure to encourage a higher-level policy to choose reachable subgoals for a lower-level policy. Because of these technical features, HAC is shown to outperform HIRO on the continuous control benchmarks. FuN [4], HIRO [3], and HAC [2] are important contributions toward the goal of developing end-to-end Feudal HRL. However, they might require long periods of training because the highest-level policy needs to adjust its output over a large continuous subgoal space. In contrast, LIDOSS simplifies the output space of the highest-level policy by using the discrete set of subgoals discovered by the integrated subgoal discovery heuristic.

Several subgoal discovery heuristics also exist in the HRL literature, such as the count-based statistics [13], [14], graph-based clustering [15]–[17], spectral analysis [18], [19], etc. These approaches perform off-line subgoal discovery, which is separated from the process of learning the HRL agent's hierarchy of policies. This causes an overhead because the agent cannot start learning to perform a task without going through a separate phase of subgoal discovery. In contrast, LIDOSS trains an agent in an end-to-end manner by integrating the subgoal discovery heuristic and learning of the hierarchy of policies.

Lastly, hierarchical learning paradigms also exist in the area of unsupervised clustering in the form of *hierarchical clustering* [20], [21], and in the area of supervised classification in the form of *hierarchical classification* [22], [23]. *Hierarchical clustering* develops a hierarchy of clusters by either repeatedly merging small clusters into a larger one (agglomeration strategy) or splitting a larger cluster into smaller ones (divisive strategy) [21]. Yildirim and Birant [21] proposed a k-linkage agglomeration strategy that evaluates the distance between two lower-level clusters by calculating the average distance between k pairs of observations, one from each cluster. The lower-level clusters are merged into the higher-level ones based on these calculated distances. This k-Linkage approach is shown to yield more stable hierarchical clusters than other standard linkage methods for intercluster distance estimations. *Hierarchical classification* is the process of classifying a given data into classes that are organized into a hierarchy. A limitation of the standard hierarchical classification is that the exact hierarchy of the class labels needs to be predefined for the data instances, which makes incremental classification hard. Recently, Park and Kim [23] addressed

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE I
NOTATIONS AND THEIR DEFINITIONS

Notation	Definition
$\mathbb{S}$	State space
$\mathbb{A}$	Primitive action space
$\mathbb{G}$	Goal space
G	General notation for a goal in the goal space $\mathbb{G}$
$\mathbb{G}^{sub}$	Subgoal space
g	General notation for a subgoal in the subgoal space $\mathbb{G}^{sub}$
ϕ	Abstraction function for mapping a state to a goal, that is, $G = \phi(s)$; where $s \in \mathbb{S}$
ϕ^{sub}	Abstraction function for mapping a state to a subgoal, that is, $g = \phi^{sub}(s)$; where $s \in \mathbb{S}$
π^{HL}	Highest-level policy of a general Feudal HRL agent
$\pi\epsilon^{HL}$	Highest-level policy of the LIDOSS agent, which is an ϵ-greedy policy
π^{HL-1}	Intermediate-level policy of a general Feudal HRL agent (same for LIDOSS)
π^{LL}	Lowest-level policy of a general Feudal HRL agent (same for LIDOSS)
$\mathcal{G}^{sub}_{SDM}(G)$	Discrete salient subgoal set discovered by LIDOSS, containing subgoals of G
g^{HL}	Subgoal chosen by π^{HL} from $\mathbb{G}^{sub}$ (in general) or by $\pi\epsilon^{HL}$ from $\mathcal{G}^{sub}_{SDM}(G)$ (in case of LIDOSS)
g^{HL-1}	Subgoal chosen by π^{HL-1} from $\mathbb{G}^{sub}$
Q^{HL}	Highest-level Q-value function of a general Feudal HRL agent (same for LIDOSS)
Q^{HL-1}	Intermediate-level Q-value function of a general Feudal HRL agent (same for LIDOSS)
Q^{LL}	Lowest-level Q-value function of a general Feudal HRL agent (same for LIDOSS)
R_{in}	Internal reward used by a general Feudal HRL agent (same for LIDOSS) to learn Q^{HL-1}, Q^{LL}, π^{HL-1}, and π^{LL}
c^{HL}	Time period for which a highest-level subgoal g^{HL} is repeated before choosing a new one
c^{HL-1}	Time period for which an intermediate-level subgoal g^{HL-1} is repeated before choosing a new one
q	The quantization size for the subgoal space quantization in the subgoal discovery heuristic of LIDOSS (in SDM; subsection IV-B)
q^G	The quantization size for the goal space quantization in the subgoal discovery heuristic of LIDOSS (in SDM)
$\|LMX\|$	The size of Local Max (LMX) kernel used by the subgoal discovery heuristic of LIDOSS (in SDM)
ψ_{th}	The threshold *occurrence probability* of a quantized subgoal within LMX for it to be considered as salient subgoal (in SDM)
SRR	SDM Refresh Rate. The SDM updates the salient subgoal sets after every SRR^{th} episode

this problem of incremental hierarchical classification using hierarchically stacked modules of fuzzy adaptive resonance theory-supervised predictive mapping (ARTMAP) [15]. The module at each level assigns the class at that level to a new data instance and also decides whether to perform classification at the higher level. In this way, the stacked ARTMAP performs incremental hierarchical classification.

III. PRELIMINARIES

The concepts outlined in the Sections III-A–III-C are crucial to understand the design of our method LIDOSS (in Section IV) and the results of the experiments (in Section V). A list of notations used in various sections of this article is provided in Table I.

A. Universal Markov Decision Process

We consider a problem setting in which an agent is given a goal that should be reached within a limited amount of time. Such a problem can be specified to the agent in the form of a universal Markov decision process (UMDP) [2]. A UMDP consists of a state space $\mathbb{S}$, an action space $\mathbb{A}$, a goal space $\mathbb{G}$, a reward function $R(s, a|G)$ defined with respect to a goal $G \in \mathbb{G}$, a state $s \in \mathbb{S}$, and an action $a \in \mathbb{A}$. The solution of a UMDP is a goal-conditioned policy $\pi(a|s, G)$: $\mathbb{S} \times \mathbb{G} \rightarrow \mathbb{A}$, which takes a state and a goal as inputs and chooses an action $a \in \mathbb{A}$ as the output. This policy is obtained by maximizing a goal-conditioned Q-value function [24], which is defined as follows: $Q^{\pi(a|s,G)}(s, a|G) = \mathbb{E}_{a_t \sim \pi(a_t|s_t,G)}[\sum_{t=0}^{t=T^s_G} \gamma^t R(s_t, a_t|G)|s_0 = s,\ a_0 = a]$, where $\gamma \in (0, 1]$ is a discount factor and T^s_G is the time-horizon over which G is reached from the start state s. We use T_G to denote the average time-horizon over which G is achieved from different start states.

In this article, the goal space $\mathbb{G}$ is defined as a lower-dimensional abstraction of the state space $\mathbb{S}$, obtained using a predefined domain-specific[2] abstraction function $\phi(s)$. This function can map a set of states to the same goal. Technically, *a goal represents a set of target states, any of which the agent needs to reach*.

All the components of the UMDP discussed above, including the goal, are externally specified to the agent as part of the goal-reaching problem. Henceforth, we refer to the externally specified UMDP as the ***core*** UMDP. The externally specified goal is referred to as ***exogenous*** goal *wherever it is necessary to* clearly distinguish it from any goal or subgoal internally chosen by the agent for learning or subgoal discovery, discussed later.

[2]Specific to a particular task domain (refer to Section V).

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

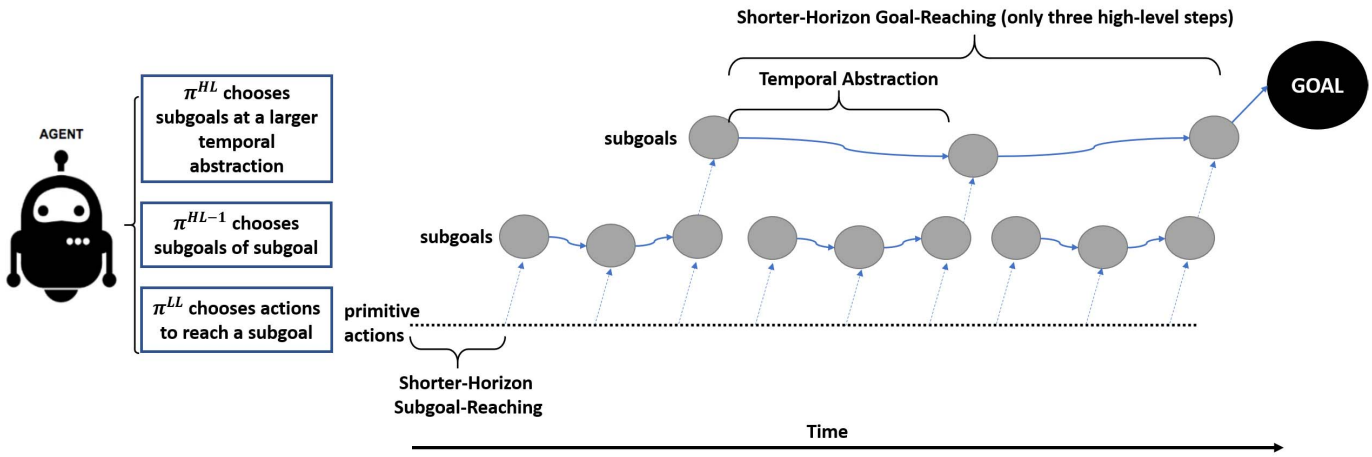


Fig. 1. Example of Feudal HRL, with larger temporal abstraction at the highest level and shorter-horizon (reachable) subgoals at the lowest level.

B. Feudal HRL and Temporal Abstraction

In the scope of this article, Feudal HRL [2], [8] can be formally defined in terms of a hierarchy of UMDPs as follows:

Given a *core* UMDP, the Feudal HRL agent internally decomposes it into a hierarchy of subgoal-based UMDPs. The agent has two internal components: a subgoal space $\mathbb{G}^{\text{sub}}$ and a reward function $R_{\text{in}}(s, a|g)$ defined with respect to a subgoal $g \in \mathbb{G}^{\text{sub}}$. Similar to the goal space, the subgoal space is also defined as a lower-dimensional abstraction of the state space. A state $s \in \mathbb{S}$ can be mapped to a corresponding lower-dimensional subgoal in $\mathbb{G}^{\text{sub}}$ through a predefined domain-specific abstraction function $\phi^{\text{sub}}(s)$. The abstraction function can map a set of states to the same subgoal. Technically, *a subgoal represents a set of intermediate states between the current state and the goal*. The internal reward $R_{\text{in}}(s, a|g)$ is domain-specific, but generally defined as greater than or equal to zero only in the states near the subgoal, otherwise it has a negative value [2].

The highest-level UMDP in the hierarchy is denoted as $l = \text{HL}$. Its state space and goal space are the same as those of the core UMDP. However, its action space is the subgoal space $\mathbb{G}^{\text{sub}}$. The highest-level policy, denoted as $\pi^{\text{HL}}(g^{\text{HL}}|s, G)$, takes a state $s \in \mathbb{S}$ and an exogenous goal $G \in \mathbb{G}$ as the inputs, and chooses a subgoal $g^{\text{HL}} \in \mathbb{G}^{\text{sub}}$ as its action.[3] The chosen subgoal is repeated for c^{HL} time steps or until it is reached, before a new subgoal can be chosen (refer to Section III-C to understand how this is implemented). c^{HL} is greater than one but less than T_G. Henceforth, we refer to the chosen subgoal as the *output* of π^{HL} rather than *action*, to avoid confusion with the per-step action taken by the agent in the core UMDP. The reward used to train the highest-level policy is defined as $R^{\text{HL}}(s_t, g^{\text{HL}}|G) = \sum_{i=0}^{i=c^{\text{HL}}} \gamma^i R(s_{t+i}, a_{t+i}|G)$. Here, $R(s, a|G)$ is the reward function of the core UMDP (Section III-A) and $a_t \in \mathbb{A}$ is an action taken by the agent in the core UMDP at time t, using the lowest-level policy described below. The Q-value is denoted as $Q^{\text{HL}}(s, g^{\text{HL}}|G)$.

[3]The general notation for a subgoal is $g \in \mathbb{G}^{\text{sub}}$. The superscript in g^{HL} is used to clarify that the subgoal is chosen by the highest-level policy.

The intermediate-level UMDP in the hierarchy is denoted as $l = \text{HL} - 1$. Its state space is the same as that of the core UMDP. However, both the goal space and the action space are same as the subgoal space $\mathbb{G}^{\text{sub}}$. The policy at this level, denoted as $\pi^{\text{HL}-1}(g^{\text{HL}-1}|s, g^{\text{HL}})$, takes the output subgoal g^{HL} of the highest-level policy π^{HL} as an input along with the state $s \in \mathbb{S}$, and chooses a subgoal $g^{\text{HL}-1} \in \mathbb{G}^{\text{sub}}$ as the action. Henceforth, we refer to the chosen subgoal as the *output* of $\pi^{\text{HL}-1}$ rather than the *action*. The output $g^{\text{HL}-1}$ is basically *a subgoal of the highest-level subgoal*. The chosen subgoal is repeated for $c^{\text{HL}-1}$ time steps or until it is reached, before a new subgoal can be chosen. $c^{\text{HL}-1}$ is greater than one but less than c^{HL}. The reward used to train the intermediate-level policy is defined as $R^{\text{HL}-1}(s_t, g^{\text{HL}-1}|g^{\text{HL}}) = \sum_{i=0}^{i=c^{\text{HL}-1}} \gamma^i R_{\text{in}}(s_{t+i}, a_{t+i}|g^{\text{HL}})$, where $R_{\text{in}}(s, a|g^{\text{HL}})$ is the internal reward function of the agent, defined earlier. The Q-value is denoted as $Q^{\text{HL}-1}(s, g^{\text{HL}-1}|g^{\text{HL}})$.

The lowest-level UMDP in the hierarchy is denoted as $l = \text{LL}$. Its state space and the action space are the same as those of the core UMDP. However, its goal space is the subgoal space $\mathbb{G}^{\text{sub}}$. The policy at this level, denoted as $\pi^{\text{LL}}(a|s, g^{\text{HL}-1})$, takes the output subgoal $g^{\text{HL}-1} \in \mathbb{G}^{\text{sub}}$ of the intermediate-level policy as an input along with the state $s \in \mathbb{S}$, and chooses an action $a \in \mathbb{A}$. Henceforth, we refer to a as a *primitive* action. The repetition time of the chosen action is $c^{\text{LL}} = 1$, that is, the agent takes a primitive action at every time step. The reward used to train the lowest-level policy is $R^{\text{LL}}(s_t, a_t|g^{\text{HL}-1}) = R_{\text{in}}(s_t, a_t|g^{\text{HL}-1})$. The Q-value is denoted as $Q^{\text{LL}}(s, a|g^{\text{HL}-1})$.

1) Temporal Abstraction in Feudal HRL: When the exogenous goal is distant from all the start states in a core UMDP, it poses the long-horizon goal-reaching problem, which can be difficult to solve using standard reinforcement learning approaches [4], [25]. Feudal HRL addresses this problem through *temporal abstraction* [26] achieved using the subgoals. Temporal abstraction at the highest level of the hierarchy means that each *individual* decision step of the policy π^{HL} (that is, choosing a subgoal g^{HL}) occurs after multiple

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

primitive action steps (equal to c^{HL} steps). This reduces the long-horizon goal-reaching problem into a shorter-horizon problem at the highest level, because the policy π^{HL} needs to learn a sequence of subgoals that is much shorter than the sequence of primitive actions (see the example in Fig. 1). At the same time, the intermediate-level policy $\pi^{\mathrm{HL}-1}$ takes the highest-level subgoal as the target and chooses a sequence of subgoals, which provides temporal abstraction at the intermediate level (Fig. 1). Lastly, the lowest-level policy π^{LL} takes an intermediate-level subgoal as the target to choose the primitive actions, which poses a shorter-horizon learning problem at the lowest level (Fig. 1). This hierarchical decomposition of a long-horizon goal-reaching problem into the shorter-horizon problems can potentially improve the learning of the goal-reaching behavior [2]–[4].

2) Trade-Off Between Temporal Abstraction and Subgoal Reachability: The benefit of temporal abstraction could be obtained using a two-level hierarchy with just π^{HL} and π^{LL}. However, larger temporal abstraction at the highest level means that the lowest-level policy π^{LL} must take a longer sequence of primitive actions to reach a subgoal. Such a subgoal might become unreachable within a limited time when the lowest-level policy is untrained. On the contrary, the intermediate-level policy $\pi^{\mathrm{HL}-1}$ in a three-level hierarchy might resolve this trade-off. In this case, the highest-level policy π^{HL} can choose a subgoal g^{HL} at a larger temporal abstraction but that subgoal is not given to the lowest-level policy π^{LL} as an input. Rather, $\pi^{\mathrm{HL}-1}$ takes that subgoal as input and chooses a *subgoal of subgoal* $g^{\mathrm{HL}-1}$, which requires fewer primitive actions to be reached using π^{LL} (see Fig. 1). Hence, subgoal reachability is maintained at the lowest level. This trade-off between temporal abstraction and subgoal reachability is important for analyzing the construction of our approach (Section IV) and the results of the experiments (Section V).

C. Base Framework: Hierarchical Actor Critic

HAC [2] is a state-of-the-art framework for end-to-end training of a Feudal HRL agent. HAC provides both temporal abstraction and subgoal reachability, as discussed further below. It uses an actor-critic [27] combination at each level of the hierarchy of policies. The critic at level l is a Q-value function Q^l,which is learned by *minimizing* the following squared error [27]:

$$(Q^l(s_i, g^l|g^{l+1}; \theta^{Q^l}) - [R^l + \gamma^{c^l} Q^l(s_{i+c^l}, \hat{g}^l|g^{l+1}; \theta^{Q^l}])^2 \tag{1}$$

where $\hat{g}^l$ is the next subgoal chosen by π^l and θ^{Q^l} are the parameters of the Q-value function. The actor is a policy π^l, which is learned using the following policy gradient [27]:

$$\nabla_{g^l} Q^l(s_i, g^l|g^{l+1}) \nabla_{\theta^{\pi^l}} \pi^l(g^l|s_i, g^{l+1}; \theta^{\pi^l}). \tag{2}$$

Here, g^l is the output of π^l in a state s_i and θ^{π^l} are the parameters of the policy π^l. For $l = \mathrm{HL}$, $g^{l+1} = G \in \mathbb{G}$, that is, the exogenous goal. For $l = \mathrm{LL}$, the output $g^l = a \in \mathbb{A}$, that is, the primitive action. Equation (2) implies that the output of the policy π^l is incrementally shifted in the continuous subgoal space until it converges on a region where the Q-value is maximum and the gradient $\nabla_{g^l} Q^l(s_i, g^l|g^{l+1})$ tends to zero.

1) How Does HAC Achieve Temporal Abstraction?: As per (2), the actor at each level (above the lowest-level) is trained to choose a subgoal that has a larger Q-value, from the continuous subgoal space. In a continuous subgoal space, the subgoals closer to the goal have a higher Q-value as compared to the subgoals closer to the current state, especially if the rewards are greater than or equal to zero *only near* the goal states. This naturally encourages the actor to choose the subgoals that are farther from the current state and closer to the goal, hence requiring multiple primitive actions to reach and achieving temporal abstraction.

2) How Does HAC Maintain Subgoal Reachability?: HAC maintains subgoal reachability using a procedure called *subgoal testing*. HAC tests if a subgoal g^l chosen by a higher-level policy π^l is achieved within c^l time steps or not, where c^l is the designated subgoal repetition time. If the subgoal is not achieved within c^l time steps, the policy π^l receives a *penalty* (negative reward) proportional to c^l, for choosing that subgoal. This encourages π^l to learn to choose those subgoals that can be reached within c^l time steps.

Other features of HAC are discussed in Appendix *A* of the Supplementary Material. Our method, discussed in Section IV, uses HAC as the base framework. Therefore, it uses the above-mentioned features of HAC.

IV. LIDOSS: HRL With Integrated Discovery of Salient Subgoals

This section presents the details of our method LIDOSS, including the composition of the LIDOSS agent (in Section IV-A) and the heuristic used for subgoal discovery (in Section IV-B). A list of advantages and limitations of LIDOSS is provided in the Conclusion (Section VI).

A. LIDOSS Agent Composition

The LIDOSS agent shares the following components with the HAC framework: the lower-level policies (actors) $\pi^{\mathrm{HL}-1}$ and π^{LL}, the lower-level Q-value functions (critics) $Q^{\mathrm{HL}-1}$ and Q^{LL}, their input space, output space, and the learning rules [(1) and (2)] are the same as in HAC. LIDOSS also uses the *subgoal testing* (Section III-C) at all the levels to learn to choose reachable subgoals.

LIDOSS differs from HAC by integrating the subgoal discovery at the highest level, using a nonparametric discrete-action policy as the highest-level policy, and using the discrete set of discovered subgoals as the outputs of that highest-level policy. These aspects of LIDOSS are as described below.

In HAC, the highest-level policy π^{HL} incrementally shifts its output over the continuous subgoal space using the policy parameter update ($\nabla_{\theta^{\pi^{\mathrm{HL}}}}$) based on the gradient of the Q-value of the current output ($\nabla_{g^{\mathrm{HL}}} Q^{\mathrm{HL}}(s_i, g^{\mathrm{HL}}|G)$) (equation 2). This can be interpreted as the highest-level policy searching for an optimal subgoal over the large continuous subgoal space. On the contrary, LIDOSS uses a subgoal discovery module (SDM) to discover a *sparse and discrete*

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

set of salient subgoals from the continuous subgoal space $\mathbb{G}^{\text{sub}}$, which can be used as the *discrete* outputs of the highest-level policy. This effectively reduces the search space of the highest-level policy by restraining it to the discovered salient subgoals.

Furthermore, LIDOSS uses a nonparametric discrete-action ϵ − greedy policy at the highest level, denoted as $\pi\epsilon^{\text{HL}}(g^{\text{HL}}|s, G)$. It chooses the subgoal with the largest Q-value with probability $1-\epsilon$ and chooses a random subgoal with probability ϵ (for exploration), from the set of the discovered salient subgoals. This also reduces the overhead of learning a parametric continuous-action policy, such as in HAC.

The Q-value functions at all levels, including the highest level, are implemented using single-output Q networks. The highest-level Q network takes a subgoal as an input along with a state and exogenous goal, and predicts the Q-value of that subgoal (as also mentioned in Section I). The learning rule for the highest-level Q-value function is same as in (1), with the subgoals chosen by $\pi\epsilon^{\text{HL}}$.

SDM is described in Section IV-B. It *incrementally* extracts the salient subgoals from the continuous subgoal space $\mathbb{G}^{\text{sub}}$ by using a probability-based subgoal discovery heuristic. The discrete set of salient subgoals corresponding to an exogenous goal G is denoted as $\mathcal{G}^{\text{sub}}_{\text{SDM}}(G)$. At the beginning of each episode, the LIDOSS agent fetches the subgoal set $\mathcal{G}^{\text{sub}}_{\text{SDM}}(G)$ from SDM, for the exogenous goal G given in that episode. The goal itself is added to this set, making the set of outputs of $\pi\epsilon^{\text{HL}}$ equal to $\mathcal{G}^{\text{sub}}_{\text{SDM}}(G)+\{G\}$. Thus, if the salient subgoals are not available initially, the intermediate-level policy $\pi^{\text{HL}-1}$ takes the exogenous goal as the input subgoal. This usually happens in the initial stages of training because the subgoal discovery in SDM is an incremental process. When the salient subgoals are available in $\mathcal{G}^{\text{sub}}_{\text{SDM}}(G)$, the policy $\pi\epsilon^{\text{HL}}$ chooses a subgoal from $\mathcal{G}^{\text{sub}}_{\text{SDM}}(G) + \{G\}$, which is then given as the input to $\pi^{\text{HL}-1}$.

In the next section, we discuss how the SDM discovers a set of salient subgoals.

B. LIDOSS Subgoal Discovery

The subgoal discovery heuristic used in LIDOSS searches for the subgoal states that appear more frequently than others on the state-transition trajectories[4] connecting various start states to the goal(s). It selects such subgoals as the *salient subgoals*, which are then used as the outputs of the highest-level policy of the LIDOSS agent (Section IV-A). The frequency of occurrence of any subgoal g on the trajectories leading to a goal G is represented by an *occurrence probability* $p(g|G)$, which is estimated using the episodic data gathered by the agent during exploration and training. The intuition behind the proposed heuristic is based on the following assumption,

Assumption 1: a subgoal g with a greater occurrence probability $p(g|G)$ recorded after multiple episodes is more likely to appear on various possible trajectories leading to the goal G in the future episodes as well. Hence, it is one of the *salient subgoals* and it can be used as one of the outputs of the highest-level policy of LIDOSS agent. That policy can choose g as an intermediate target to reach the goal in the future. This assumption also holds if the behavior of the agent changes in the future as a result of training or stochasticity, leading to the discovery of new salient subgoals. In such a case, g can still be retained in the salient subgoal set without directly affecting the highest-level policy, because policy chooses an output from the salient subgoal set *based on the latest Q-values* (refer to Section IV-A).

The proposed subgoal discovery heuristic is designed to achieve the following properties without using the rewards or the subgoal testing penalty (Section III-C) that are used to train the policies:

Bottleneck Subgoal Discovery: bottlenecks are those states in the state space where several trajectories leading to the goal(s) converge [13], [28]. The implication is that the topology of the state space forces the agent to go through the bottleneck states to reach the goal(s). Such bottlenecks increase the difficulty of reaching the long-horizon goal(s). The probability-based subgoal discovery in LIDOSS is intended to find the subgoals corresponding to such bottleneck states (remember that a subgoal is an abstraction of a state; Section III-B).

Subgoal Reachability: as discussed in Section III-B, subgoal reachability is important in the end-to-end HRL. Hence, the subgoal discovery heuristic must select the salient subgoals in such a manner that the subgoal reachability is maintained. This is achieved by comparing the $p(g|G)$ values of different subgoals within local neighborhoods in the subgoal space, rather than comparing them globally. This is called Local Max Saliency determination, described later in Section IV-B2. Selecting the subgoals with globally larger $p(g|G)$ might result in most of the discovered subgoals being concentrated only around the bottlenecks. If the bottleneck subgoals are themselves difficult to reach during the early stages of training, then the performance of the agent might suffer. The heuristic of LIDOSS selects at most one subgoal from each local neighborhood as the salient subgoal through the local comparison of $p(g|G)$ values. This ensures that the discovered salient subgoals are not over-concentrated in a single neighborhood, such as around the bottlenecks, but more widely distributed over the subgoal space to maintain the local reachability of the subgoals.

The subgoal discovery heuristic is further explained below. A corresponding pseudocode is provided later in Algorithm 1.

1) Procedure of Subgoal Discovery: The procedure of subgoal discovery involves multiple stages of operations carried out *after every episode*. The operations are depicted as part of the SDM in Fig. 2. First, the state-transition trajectory taken by the agent during an episode is received by the SDM. The terminal state in this trajectory is abstracted as a *proxy* goal $G \in \mathbb{G}$ using the predefined goal space abstraction function $\phi(s)$. Using the terminal states as the *proxy* goals helps the SDM discover multiple sets of subgoals corresponding to a variety of goals and also avoid wasting an episodic data if the exogenous goal is not achieved in that episode. Furthermore, each state in the trajectory is mapped to a corresponding subgoal in the

[4]A state-transition trajectory is the sequence of states observed during an episode

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

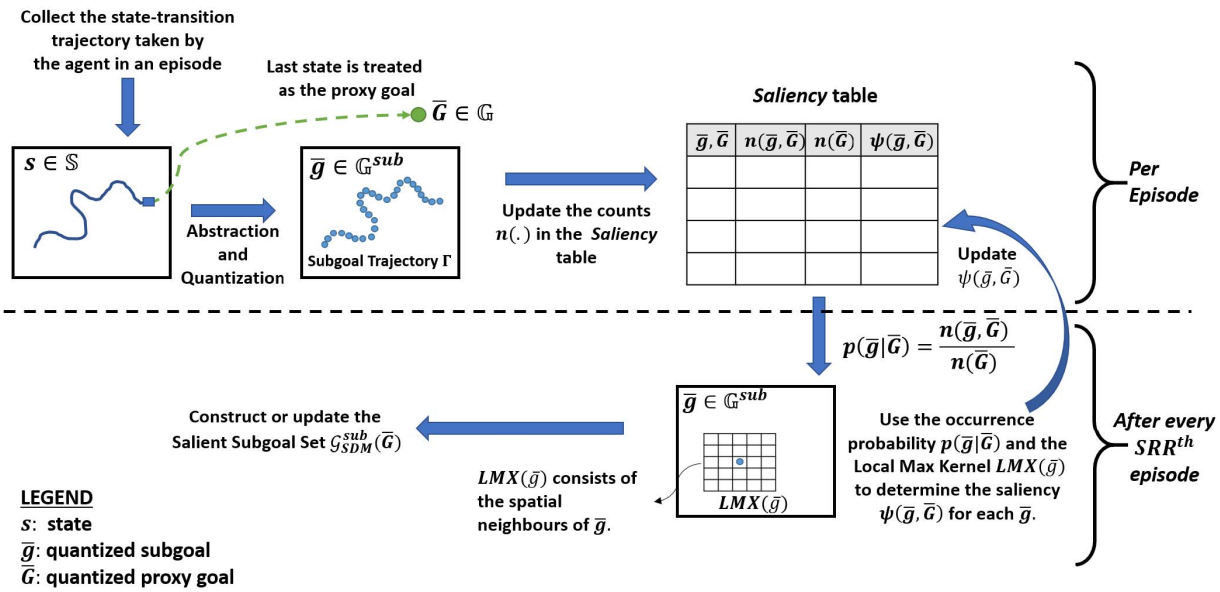


Fig. 2. Flow of operations in the SDM.

subgoal space $\mathbb{G}^{\text{sub}}$ through the predefined subgoal abstraction function $\phi^{\text{sub}}(s)$. The trajectory consisting of the subgoals is denoted as Γ_g.

Next, the SDM quantizes the proxy goal and the subgoals present in Γ_g. Quantization is performed to enable the counting (across multiple episodes) of the occurrence of the proxy goal, and the occurrence of subgoals on the trajectories leading to the proxy goal. In a continuous space, the exactly same proxy goal or subgoal may not be encountered twice in different episodes [29]. This could make the count of every subgoal and proxy goal equal to one, leading to equal occurrence probabilities and failure of subgoal discovery. Quantization ensures that the occurrences of the subgoals (or proxy goals) that are very close to each other are added into the occurrence of a single quantized subgoal (or quantized proxy goal). The quantization is performed as follows:

$$\bar{g} = \left\lfloor \frac{g}{q} \right\rfloor \times q \tag{3}$$

for the subgoal, and

$$\bar{G} = \left\lfloor \frac{G}{q^G} \right\rfloor \times q^G \tag{4}$$

for the goal. Here, $\bar{g}$ and $\bar{G}$ are the quantized subgoal and proxy goal, respectively. q and q^G are the quantization sizes. The notation $\lfloor \cdot \rfloor$ indicates the *floor* function. The quantization sizes are kept very small such that the quantized subgoals and proxy goals do not violate the topology of the subgoal space $\mathbb{G}^{\text{sub}}$ and the goal space $\mathbb{G}$, respectively (refer to Appendix *C* of the Supplementary Material). The new quantized subgoal trajectory is denoted as $\Gamma'_{\bar{g}}$.

After quantization, SDM updates a *Saliency Table* for the entries corresponding to different pairs $(\bar{g}, \bar{G})$. The *Saliency Table* consists of the following entries: 1) *$(\bar{g}, \bar{G})$ pair*, which is the *key* of a row; 2) $n(\bar{G})$, which is the count of occurrence of a quantized proxy goal $\bar{G}$, incremented over multiple episodes; 3) $n(\bar{g}, \bar{G})$, which is the count of occurrence of a quantized subgoal $\bar{g}$ on the trajectories terminating in the proxy goal $\bar{G}$, incremented over multiple episodes; and 4) $\psi(\bar{g}, \bar{G})$, which is the saliency value of $\bar{g}$ with respect to $\bar{G}$. Then, **the occurrence probability is calculated as** $p(\bar{g}|\bar{G}) = ((n(\bar{g}, \bar{G}))/(n(\bar{G})))$.

In this manner, SDM maintains the counts of the quantized subgoals and quantized proxy goals over multiple episodes. SDM updates the salient subgoal sets, denoted as $\mathcal{G}^{\text{sub}}_{\text{SDM}}(\bar{G})$, corresponding to different quantized proxy goals stored in the *Saliency Table*, after every SRRth episode. Here, SRR stands for SDM refresh rate. The update procedure is described in the Sections IV-B2 and IV-B3.

2) Local Max Saliency: As mentioned earlier, the subgoal discovery heuristic selects the salient subgoals within local neighborhoods such that their reachability is maintained. SDM determines the saliency of a quantized subgoal by comparing its probability values with its local neighbor quantized subgoals. Local comparisons ensure that if the occurrence probability of a quantized subgoal is higher than its neighbors and passes a threshold, it is selected as a salient subgoal even if it does not have a globally higher occurrence probability.

The saliency of a quantized subgoal is determined using (5), defined below. In this equation, LMX$(\bar{g})$ is a Local Max kernel, which consists of the neighboring quantized subgoals around $\bar{g}$ (Fig. 2). The number of neighbors in this kernel is denoted as |LMX|. We use a spatial neighborhood to constitute LMX. Then, $\max_{\text{LMX}(\bar{g})} = \max_{\bar{g}' \in \text{LMX}(\bar{g})} p(\bar{g}'|\bar{G})$ is the maximum occurrence probability among the neighbors of $\bar{g}$, with respect to a proxy goal $\bar{G}$. If the saliency $\psi(\bar{g}, \bar{G}) = 1$, then $\bar{g}$ is considered as a salient subgoal with respect to $\bar{G}$

$$\psi(\bar{g}, \bar{G}) = \begin{cases} 0, & \text{if } p(\bar{g}|\bar{G}) < \max_{\text{LMX}(\bar{g})} \\ 0, & \text{if } \max_{\text{LMX}(\bar{g})} \\ & \quad <= p(\bar{g}|\bar{G}) < \psi_{\text{th}} \\ 1, & \text{otherwise.} \end{cases} \tag{5}$$

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Equation (5) implies that a quantized subgoal $\bar{g}$ is considered salient with respect to a quantized proxy goal $\bar{G}$ only if it passes two conditions. The first condition is that the probability $p(\bar{g}|\bar{G})$ should not be less than the maximum probability among the neighboring quantized subgoals (that is, $\max_{\mathrm{LMX}(\bar{s})}$). The second condition is that even if $p(\bar{g}|\bar{G})$ is greater than or equal to the maximum probability among the neighbors, it should not be less than ψ_{th}, which is a threshold probability value called *saliency threshold*. If there are multiple states within $\mathrm{LMX}(\bar{g})$ with nonzero saliency, only one of them is chosen as the salient subgoal.

3) Updating the Salient Subgoal Sets: The salient subgoal set $\mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G})$ corresponding to *each* quantized proxy goal $\bar{G}$ in the *Saliency Table* is updated as follows:

$$\mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G}) = \begin{cases} \mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G}) + \{\bar{g}\}, & \textbf{if } \psi(\bar{g}, \bar{G}) = 1 \\ & \quad \wedge\, \bar{s} \notin \mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G}) \\ \mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G}) - \{\bar{g}\}, & \textbf{if } \psi(\bar{g}, \bar{G}) = 0 \\ & \quad \wedge\, \bar{s} \in \mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G}) // \text{pruning.} \end{cases} \tag{6}$$

Equation (6) implies that a quantized subgoal $\bar{g}$ is added to the subgoal set $\mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G})$ if it is salient ($\psi(\bar{g}, \bar{G}) = 1$) and not already present in the set. On the contrary, $\bar{g}$ is removed from $\mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G})$ if it is present in the set but not salient anymore. This is called pruning. This update operation for each $\bar{G}$ in the *Saliency Table* is performed after every $\mathrm{SRR}^{\mathrm{th}}$ episode.

As mentioned in Section IV-A, the LIDOSS agent gets the salient subgoal set corresponding to an exogenous goal at the beginning of every episode and uses that as the output space of the highest-level policy. If the nonquantized exogenous goal is G, the salient subgoal set is obtained as $\mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(G) = \mathcal{G}^{\mathrm{sub}}_{\mathrm{SDM}}(\bar{G})$, if $\bar{G} = \lfloor (G/q^G) \rfloor \times q^G$, as per equation 4.

4) Remarks on the Subgoal Discovery Heuristic: The subgoal discovery heuristic, described above, does not use the rewards from the core UMDP. Hence, it is difficult to provide a theoretical analysis of the heuristic from the policy optimality perspective. The heuristic is designed to work without reward-based supervision and to find salient subgoal states that include topological bottlenecks in the state space. Hence, the subgoal discovery uses the occurrence probability of the subgoals to identify the salient subgoals. The efficacy of this heuristic is *empirically* shown in the task domains, which contain bottlenecks (refer to the experiments discussed in Section V). Hence, LIDOSS is more suitable than a simple *uniform* sampling of subgoals if the task domain contains bottlenecks.

Moreover, subgoal discovery occurs continuously as long as new data are made available through the exploration performed by the agent. Hence, the set of outputs of the highest-level policy of LIDOSS keeps changing dynamically. This did not pose a practical issue in the task domains used in our experiments (Section V), in which the subgoal space is bounded and not very large, because new subgoals are not discovered after the agent has completely explored the subgoal space. However, learning may not be stable in other task domains with a very large or infinite subgoal space.

Algorithm 1 Subgoal Discovery Procedure of LIDOSS

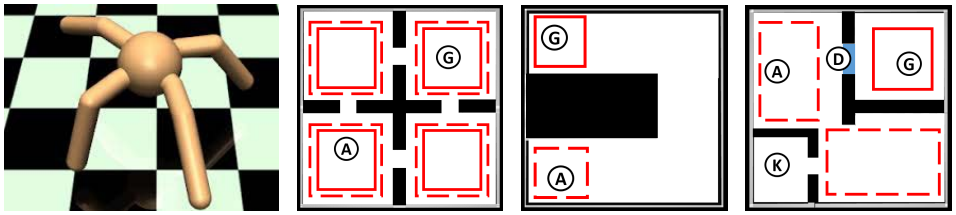
```
Initialize q, q^G, SRR, |LMX|, ψ_th (refer to Table I)
Initialize n_table ← 0    ▷ Size of Saliency Table Initialize
nTrain ← number of training episodes
foreach EP ∈ [0, nTrain) do
    Run the training episode
    Save the state-transition trajectory Γ_s at the end of the
    episode
    Initialize subgoal trajectory Γ_g ← ∅
    foreach s ∈ Γ_s do
        Insert subgoal φ^sub(s) in Γ_g                ▷ Abstraction
    s_T ← terminal state in Γ_s.
    Proxy goal G ← φ(s_T)                             ▷ Abstraction
    Quantize proxy goal: Ḡ ← ⌊G/q^G⌋ × q^G            ▷ Quantization
    Initialize quantized subgoal trajectory Γ'_ḡ ← ∅
    prev ← ∅
    foreach g ∈ Γ_g do
        Quantize subgoal: ḡ ← ⌊g/q⌋ × q               ▷ Quantization
        if ḡ ≠ prev    ▷ avoid consecutive duplicates then
            Insert ḡ in Γ'_ḡ
            prev ← ḡ
                    ▷ Time complexity of quantization: O(|Γ_g|)
    foreach ḡ ∈ Γ'_ḡ do
        if (ḡ, Ḡ) ∉ Saliency Table then
            Insert (ḡ, Ḡ) in Saliency Table.
            n_table ← n_table + 1
            Set n(Ḡ) ← 0, n(ḡ, Ḡ) ← 0, ψ(ḡ, Ḡ) ← 0
        n(Ḡ) ← n(Ḡ) + 1                               ▷ Count increment
        n(ḡ, Ḡ) ← n(ḡ, Ḡ) + 1                         ▷ Count increment
            ▷ Time complexity of count increments: O(|Γ'_ḡ|)
    if (EP mod SRR) = 0 then
        foreach (ḡ, Ḡ) in Saliency Table do
            Initialize LMX(ḡ) ← ∅
            Get |LMX| nearest quantized subgoal neighbors
            of ḡ, add them to a temporary set temp
            foreach ḡ' ∈ temp do
                if (ḡ', Ḡ) ∈ Saliency Table then
                    Insert ḡ' into LMX(ḡ)
                p(ḡ|Ḡ) = n(ḡ, Ḡ) / n(Ḡ)
                Determine saliency ψ(ḡ, Ḡ) as per
                equation 5
                Update G^sub_SDM(Ḡ) as per equation 6
                    ▷ Time complexity of the updates:
        O(n_table × |LMX|)
▷ Average time complexity per episode: O(|Γ_g|) +
O(|Γ'_ḡ|) + O(n_table × |LMX| / SRR)
```

V. Experiments

This section describes the experiment setup to evaluate LIDOSS (Section V-A), the methods it is compared with (Section V-B), and the observed results along with their analysis (Section V-C). A discussion about the computation time required by LIDOSS and HAC is provided in Appendix *D* of the Supplementary Material.

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



(a) (b) (c) (d)

Fig. 3. MuJoCo Ant environments for continuous control tasks used in our experiments. The labels are as follows: "A" is Agent, "G" is exogenous Goal, "K" is Key, and "D" is Door. The solid-border boxes show the regions within which a goal can be specified. The dashed-border boxes show the regions within which the agent can start. (a) Ant agent. (b) Ant four rooms. (c) Ant Maze. (d) Ant Key-Lock.

A. Setup

The experiments are performed using a set of *simulated* MuJoCo continuous control task domains [9], described later. The task environments are deterministic. The tasks require goal-directed navigation, in which the agent has to traverse through the continuous state space using continuous primitive actions, to reach the target goal state(s). The agent is a simulated Ant robot with a torso and four limbs, depicted in Fig. 3(a). The state, goal, subgoal, and primitive action spaces are as follows:

The state space $\mathbb{S}$ is a continuous vector space with 29 dimensions. The first three dimensions represent the location of the Ant agent, denoted as (X, Y, Z). The 16th and 17th dimensions represent the velocity of the Ant in the directions of X and Y, respectively. The rest of the dimensions represent the orientation of Ant's torso and joints, and the velocities of the joints. The primitive action space $\mathbb{A}$ is 8-D and continuous. Various dimensions of the action space represent the torques to be applied on the Ant's limb joints.

The goal space $\mathbb{G}$ is 3-D and continuous. It is obtained using the predefined abstraction function $\phi(s)$ (refer to Section III-B), which extracts the first three dimensions from a state $s \in \mathbb{S}$ as a goal $G \in \mathbb{G}$. Therefore, a goal is a target location in the 3-D navigation environment.

The subgoal space $\mathbb{G}^{\text{sub}}$ is 5-D and continuous. The predefined subgoal abstraction function $\phi^{\text{sub}}(s)$ (refer to Section III-B) extracts the first three dimensions [location (X, Y, Z)] and the 16th and 17th dimensions (velocity) from a state $s \in \mathbb{S}$ as a subgoal $g \in \mathbb{G}^{\text{sub}}$. Thus, a subgoal is an *intermediate* location and the velocity is the location that the agent should achieve to potentially reach the goal location.

All the spaces described above are taken exactly as defined by Levy *et al.* [2] for HAC, which is used as the baseline method in the experiments.

For all experiments, the training and testing is done in *alternating* batches of 100 episodes each, similar as the procedure used by Levy *et al.* [2] for HAC. One experiment trial consists of multiple alternating training and testing batches. For each testing batch, the percentage of the episodes (out of 100) in which the agent successfully reaches the *exogenous* goal is recorded as the *success rate* in that batch. The success rate is used as the performance measure during a trial. The experiments are divided into two types: *fixed goal experiments* and *dynamic goal experiments*. In the fixed goal experiments, the exogenous goal is randomly selected from the goal space at the beginning of the first episode of a trial. The same goal is then selected for the entirety of every subsequent episode in that trial. The purpose of this setup is to test how quickly the agent learns to reach a particular goal from different *distant* start states. In the dynamic goal experiments, the exogenous goal is randomly selected at the beginning of each new episode in a single trial. The purpose of this setup is to test how quickly the agent generalizes to dynamically changing goals.

The MuJoCo task domains used for the experiments are shown in Fig. 3. The Ant Four Rooms domain [2] requires the agent (labeled "A") to navigate from one room to another to reach the exogenous goal (labeled "G"). The rooms are separated by walls and connected by passages (bottlenecks). We perform both fixed goal and dynamic goal experiments in this domain. The Ant Maze domain [3] requires the agent to navigate a "⊐"-shaped corridor to reach the exogenous goal. We perform only fixed goal experiments in this domain. The ant key-lock is our custom task domain in which the agent has to fetch a key (bottleneck, labeled "K") from one room before it can open a closed door (bottleneck, labeled "D") and reach the exogenous goal in another room. We perform only fixed goal experiments in this domain. Additionally, we also perform dynamic goal and fixed goal experiments in a very simple task domain called Ant Reacher [2], which does not contain obstacles or bottlenecks. Further details about the task domains are provided in Appendix *B* of the Supplementary Material.

The hyperparameters are the same across all tasks. The quantization sizes are $q = q^G = 0.5$ for the subgoal space and the goal space. The saliency threshold ψ_{th} is 0.3. The SRR is SRR $= 50$. The number of neighbors in the LMX kernel are $|\text{LMX}| = 24$. The ϵ value of the highest-level policy of LIDOSS is equal to 0.3 during training and 0.0 during testing. Each episode is terminated after 700 time steps. The subgoal repetition time at the highest level is $c^{\text{HL}} = 100$ time steps and at the intermediate level is $c^{\text{HL}-1} = 10$ time steps. The core UMDP reward (R) is -1 at each step and 0 only when the agent is in proximity to the exogenous goal. Similarly, the internal reward of the HRL agent (R_{in}; refer to Section III-B) is -1 at each step and 0 only when the agent is in proximity to a subgoal. Please refer to Appendix *B* of the Supplementary Material for details about how the proximity to an exogenous goal or a subgoal is determined.

As discussed in Section IV-B, the subgoal discovery heuristic of LIDOSS subsamples a discrete set of salient subgoals from the continuous subgoal space. The degree of discretization is controlled by the hyperparameters q and $|\text{LMX}|$. The

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

details about how we set these hyperparameters are provided in Appendix *C* of the Supplementary Material.

B. Methods Compared

LIDOSS is compared against the following methods in the experiments:

HAC [2]: this method is used as the standard end-to-end Feudal HRL baseline without subgoal discovery or quantization. For details, refer to Section III-C.

HAC + Quantization (HAC + Qn): this method is designed by ablating the subgoal discovery heuristic of LIDOSS and using the discrete set of subgoals, obtained only by the quantization of the continuous subgoal space (3), as the outputs of the highest-level ϵ-greedy policy. For fair comparison, the quantization size is set as $q = 0.5$ similar to LIDOSS. Quantization provides a discrete set of subgoals distributed in a uniform grid-pattern over the subgoal space [Fig. 4(a)]. HAC + Qn also works incrementally like LIDOSS, by periodically quantizing the subgoals that are the abstractions of the *explored* states, at an interval equal to the SRR of LIDOSS.

HAC + Quantization + FPS (HAC + QnFPS): this is an alternative method for salient subgoal selection. It is designed by replacing the probability-based local salient subgoal discovery heuristic of LIDOSS with the farthest point sampling (FPS) algorithm [30]. The FPS algorithm starts by randomly sampling one of the quantized subgoals. Then, it samples another quantized subgoal, which is the farthest from the first one, where the distance measure is the Euclidean distance in the subgoal space. Subsequently, it samples a third quantized subgoal, which is the farthest from the first two, and so on. In this manner, it discovers 300 subgoals[5] to form the discrete set of outputs of the highest-level ϵ-greedy policy of HAC + QnFPS agent. The FPS algorithm does not depend on the exogenous or proxy goals, hence HAC + QnFPS discovers only one common set of subgoals. The subgoals are widely distributed over the subgoal space with similar concentration in different regions [Fig. 4(b)]. Their arrangement is not directed toward bottlenecks, unlike the subgoal distribution of LIDOSS [Fig. 4(c)]. HAC + QnFPS also works incrementally by applying the FPS sampling at an interval equal to the SRR of LIDOSS.

C. Results and Discussion

The results for the fixed goal and dynamic goal experiments performed in different task domains are shown in Fig. 5. As mentioned earlier, the training and testing batches are executed alternately, where each batch is of 100 episodes. Thus, the success rate after N testing batches also indicates the performance of an agent after N training batches. All methods perform comparably in the Ant Reacher domain [Fig. 5(e)], with similar results for both dynamic goal and fixed goal setup. This indicates that neither quantization nor subgoal discovery has a beneficial effect if the topology of the environment is very simple, that is, it contains no obstacle or bottleneck, and the agent can simply navigate straight to the goal in an open space. The rest of the domains have a more complicated topology, including obstacles and bottlenecks such as doors, passages, and key (Fig. 3). In those domains, various methods show a noticeable difference in performance.

Before analyzing the results further, the following points need to be recalled from Section III: the efficacy of end-to-end HRL depends on both the *temporal abstraction* of the subgoals chosen by the highest-level policy and the *reachability* of those subgoals using the lowest-level policy. The highest-level policy learns to choose the subgoals with larger Q-values; such subgoals are farther from the current state and closer to the goal if the rewards are greater than or equal to zero only near the goal. This gradually leads to temporal abstraction during training, in combination with the repetition of a chosen subgoal for multiple time steps. At the same time, the highest-level policy also receives a large penalty for choosing a subgoal that cannot be reached using the lowest-level policy within limited time steps, which ensures the reachability of the subgoals.

Now, the performance of various methods is analyzed as follows. LIDOSS, HAC + Qn, and HAC + QnFPS achieve better *final* success rate than HAC, to varying degrees. The common aspect of these three methods is the nonparametric ϵ-greedy policy at the highest level of the HRL agent's hierarchy of policies. As discussed in Sections III-C and IV-A, the parametric HAC policy incrementally shifts its output over the continuous subgoal space, based on the gradient of the Q-value. Thus, HAC requires more training time until the highest-level policy output converges near the region of the subgoal space with larger Q-values. On the contrary, the ϵ-greedy policies of LIDOSS, HAC + Qn, and HAC + QnFPS directly choose a subgoal from the discrete set of subgoals, based on the largest Q-value (because ϵ value is zero during testing). Hence, the performance benefit for LIDOSS, HAC + Qn, and HAC + QnFPS over HAC comes from avoiding the overhead of learning a parametric continuous-action policy at the highest level. The default learning rate for the parameters of HAC's highest-level policy is 0.001. We also tested HAC by decreasing and increasing the learning rate, but it still achieved lower success rates than other methods.

1) LIDOSS Versus Pure Quantization: The results also show that quantization alone (that is, HAC + Qn) does not lead to the best gain in performance over HAC (Fig. 5). HAC + Qn uses a set of subgoals that are very densely distributed over the subgoal space [Fig. 4(a)]. In contrast, LIDOSS discovers a set of subgoals that are sparsely distributed over the subgoal space [Fig. 4(c)]. We observed that the sparse distribution provides an additional temporal abstraction effect to the LIDOSS agent. The highest-level policy of the HAC + Qn agent chooses a long and haphazard sequence of subgoals in a few episodes. This sometimes restricts the exploration done by the HAC + Qn agent, which might also affect learning. It happens because of the quantized subgoals being very close to each other. On the contrary, the highest-level policy of LIDOSS chooses subgoals that are naturally distant from each other, in terms of the number of time steps needed to reach them, because of the sparsity. This additional temporal abstraction effect helps

[5]LIDOSS discovers approximately 300 subgoals corresponding to a proxy goal in the different task domains by the end of training.

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

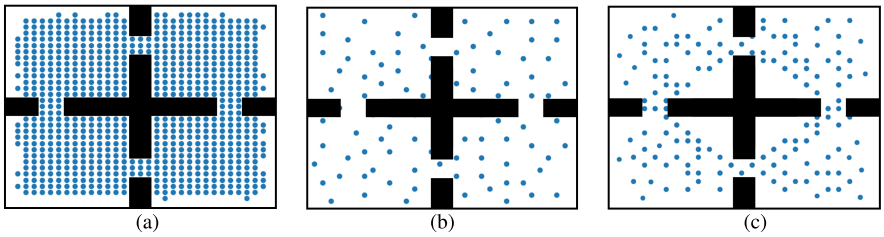


Fig. 4. Distribution of the subgoals discovered using different heuristics, in the Ant Four Rooms domain. This is a projection of the subgoals on a 2-D plane for the purpose of visualization. (a) HAC + Qn subgoals. (b) HAC + QnFPS subgoals. (c) LIDOSS subgoals.

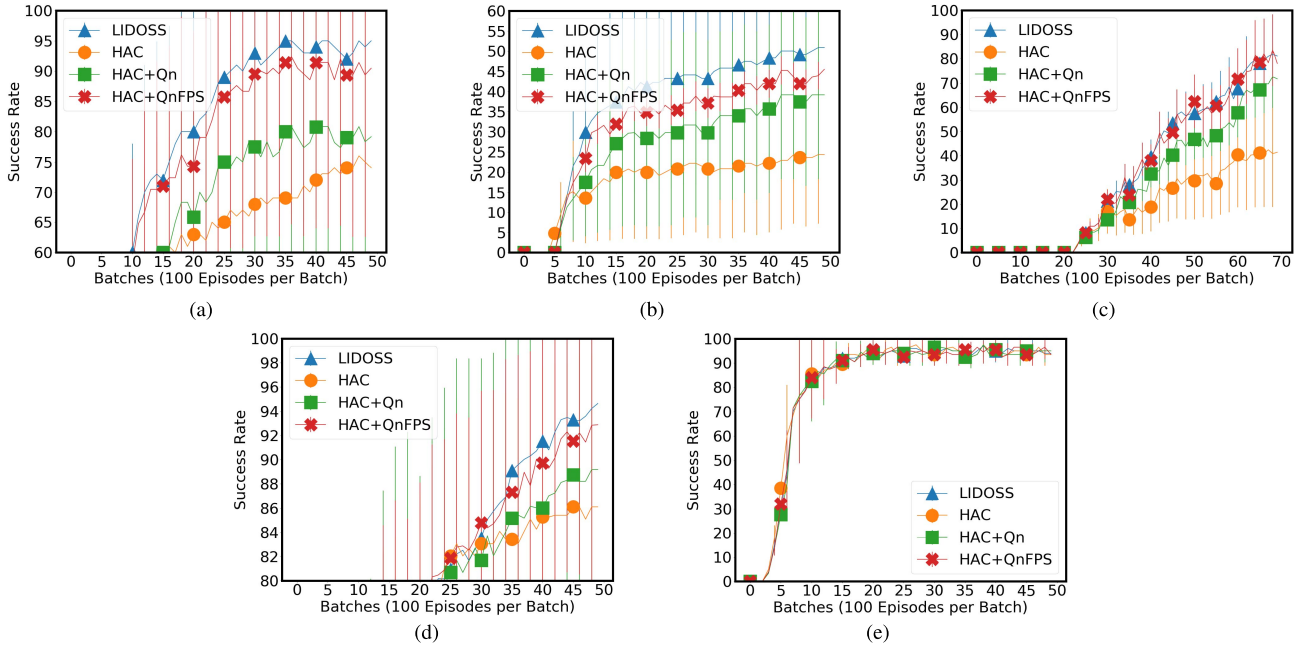


Fig. 5. Success Rates in different task domains, calculated per 100 episodes in a testing batch. The result for each method is an average of 50 trials. The x-axis shows the number of finished testing batches, which is also equal to the number of finished training batches. The scale of the y-axis is restricted to show the differences between the success rates more clearly. (a) Ant four rooms fixed goal. (b) Ant key-lock fixed goal. (c) Ant Maze fixed goal. (d) Ant four rooms dynamic goal. (e) Ant reacher dynamic/fixed goal.

in exploration and learning, leading to a better performance shown by LIDOSS.

2) LIDOSS Versus Other Subgoal Discovery Heuristics: LIDOSS performs slightly better than HAC + QnFPS in the task domains that contain bottlenecks, and are ant four rooms and ant key lock (please see Fig. 5 in this document for the experiment results). This is because HAC + QnFPS does not explicitly discover the subgoals that lie at or near the bottlenecks in the four rooms and key-lock domains, because the FPS algorithm samples the subgoals that are more uniformly distributed over the subgoal space. In those domains, we observed that the HAC + QnFPS agent gets stuck at the walls in a few cases and it is not able to traverse through the bottlenecks, whereas the LIDOSS agent chooses the bottleneck subgoals and traverses through them. However, in the ant maze domain without bottlenecks, the two methods perform comparably. Hence, LIDOSS is more suitable for the task domains containing bottlenecks.

Finally, we also tested LIDOSS after ablating the local Max kernel and only extracting the top 300 subgoals corresponding to a proxy goal, sorted by their occurrence probabilities compared globally. A majority of such subgoals are concentrated near the bottlenecks. The final performance of that agent is found to be similar to LIDOSS in the four rooms domain but worse than LIDOSS in the Key-Lock domain. The bottlenecks in the four rooms domain [Fig. 3(b)] are the passages that can be easily reached from the adjacent rooms. However, the bottlenecks in the Key-Lock domain [Fig. 3(d)] are harder to reach from certain states. For example, the Key is harder to reach from outside the key (K) room and the door to the goal room (D) is harder to reach from the key room, using a simple traversal on a straight path. In such cases, the reachability of the intermediate states also matters, which is achieved by the local selection of the salient subgoals using the local Max kernel of LIDOSS (Section IV-B).

3) Effect of the Depth of the HRL Agent's Hierarchy of Policies: As discussed in Section III-B, a hierarchy of policies with more than two levels possibly offers the advantage of greater temporal abstraction at the highest level while maintaining the reachability of the subgoals at the lowest level. On the contrary, in a two-level hierarchy, the highest-level policy has to choose the subgoals that are closer to the current state to avoid excess penalties if the agent cannot reach the subgoals (refer to *subgoal testing* in Section III-C). This

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II

FINAL SUCCESS RATE OF AN AGENT AS A FUNCTION OF THE DEPTH OF ITS HIERARCHY OF POLICIES, RECORDED AFTER 70 TESTING BATCHES IN ANT MAZE AND 50 TESTING BATCHES IN OTHER TASKS. THE 1-LEVEL AGENT IS A STANDARD RL AGENT WITH A SINGLE POLICY AND NO HIERARCHY OR SUBGOALS. THE RESULTS FOR EACH TASK ARE THE AVERAGE OF 50 TRIALS

Task	LIDOSS 3-levels	HAC 3-levels	LIDOSS 2-levels	HAC 2-levels	1-level
Four Rooms - fixed goal	94.8 $\pm$ 27.6	74.2 $\pm$ 29.5	36.3 $\pm$ 28.3	37.1 $\pm$ 30.2	6.7 $\pm$ 3.2
Four Rooms - dynamic goal	94.6 $\pm$ 28.3	85.9 $\pm$ 27.2	38.4 $\pm$ 31.1	44.7 $\pm$ 33.4	7.4 $\pm$ 3.5
Key Lock	50.6 $\pm$ 27.2	24.8 $\pm$ 18.1	8.5 $\pm$ 6.3	10.2 $\pm$ 7.1	1.1 $\pm$ 0.2
Maze	81.2 $\pm$ 14.4	41.6 $\pm$ 22.2	2.3 $\pm$ 0.4	5.5 $\pm$ 1.2	0.5 $\pm$ 0.2

causes the highest-level policy to choose a longer sequence of subgoals to reach an exogenous goal within an episode, reducing the benefit from temporal abstraction and increasing the required training time. This trade-off between temporal abstraction and subgoal reachability is most likely the reason that a 3-level HRL agent performs better than a 2-level HRL agent for both HAC and LIDOSS, as reported in Table II. We also test a 1-level nonhierarchical agent, which is a standard RL policy that takes a state and a goal of the core UMDP as the inputs, and chooses a primitive action from the continuous action space as the output. This agent does not use any subgoal, hence it has to reach the long-horizon goal using only the primitive actions. It can be seen that the 1-level agent performs the worst in all the domains.

VI. CONCLUSION

This article presents a method called LIDOSS, which integrates a subgoal discovery heuristic into the end-to-end learning of an HRL agent's hierarchy of policies. The heuristic discovers a *sparse and discrete* set of salient subgoals from a large continuous subgoal space. The salient subgoals are *simultaneously* used as the output of the highest-level policy of the LIDOSS agent. The subgoal discovery heuristic is based on the local comparison of the probabilities of occurrence of different subgoals on various state-transition trajectories leading to the goal(s). It finds a set of subgoals that includes those lying at the bottleneck regions of the state space.

LIDOSS is compared with a recently introduced end-to-end HRL method called HAC [2] in a set of continuous control tasks in the MuJoCo task domains [9]. In HAC, the highest-level policy directly uses the large continuous subgoal space as its output space, without subgoal discovery. The experiments show that LIDOSS performs better than HAC in all task domains that contain obstacles or bottlenecks. The better performance of LIDOSS is mainly because of the use of a discrete-action highest-level policy and the reduced overhead of learning a continuous-action policy, such as in HAC.

A. Advantages and Limitations

LIDOSS has the following main ***advantages***: 1) it reduces the overhead of learning a continuous-action highest-level policy over a continuous subgoal space and 2) it explicitly includes the subgoal states lying at the bottleneck regions, among other discovered subgoals, which is beneficial in the task domains containing the state-space bottlenecks. LIDOSS also has a few key ***limitations*** as follows: 1) LIDOSS might not have an advantage over other subgoal discovery heuristics, such as the FPS [30] used in our experiments, in task domains that do not contain bottlenecks; 2) LIDOSS requires quantization of the subgoal space, which might be difficult if the subgoal space is high dimensional, because of the curse of dimensionality; and 3) LIDOSS is not suitable for task domains with a very large (potentially infinite) state space because the *continuous* subgoal discovery makes the set of outputs of the highest-level policy unstable, which affects learning.

B. Future Work

To address the second limitation, we plan to replace the quantization operation with subgoal clustering using a learned subgoal similarity function in the future work. One scheme to learn such a similarity function is to label two subgoals as *similar* (label value equal to 1) if they are separated by a limited number of time steps on an episodic trajectory, otherwise, label them as *dissimilar* (label value equal to 0). A neural network can be pretrained using this labeled data in a supervised manner to predict the similarity or dissimilarity of two high-dimensional subgoals. A similar scheme is used in a recent work on topological memory-based planning [31]. During the training of the HRL agent, the pretrained similarity function can be used to find the similar subgoals and cluster them into a discrete centroid subgoal, which will provide the same effect as quantization.

LIDOSS is a model-free HRL method that requires a significant amount of data to learn the hierarchy of policies from scratch to reach new goals. In the future, we will investigate the task or goal decomposition from a different perspective, which is aligned with model-based planning [31], [32]. The agent will use the exploration data to build state-transition models at different levels of *temporal abstraction*; it will use those models to sample the subgoals and use planning to find the paths to the goal(s) through the subgoals. A model-based approach can help the agent memorize the structure or topology of the state space and subgoal space, and reduce the amount of data required to learn to reach new goals.

REFERENCES

[1] A. G. Barto and S. Mahadevan, "Recent advances in hierarchical reinforcement learning," *Discrete Event Dyn. Syst.*, vol. 13, nos. 1–2, pp. 41–77, 2003.

[2] A. Levy, G. D. Konidaris, R. Platt, Jr., and K. Saenko, "Learning multi-level hierarchies with hindsight," in *Proc. 7th Int. Conf. Learn. Represent., (ICLR)*, New Orleans, LA, USA, May 2019, pp. 1–15.

[3] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst. (NIPS)*. Red Hook, NY, USA: Curran Associates, 2018, pp. 3307–3317.

[4] A. S. Vezhnevets *et al.*, "Feudal networks for hierarchical reinforcement learning," in *Proc. 34th Int. Conf. Mach. Learn., (ICML)*, vol. 70, Aug. 2017, pp. 3540–3549.

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[5] N. Dilokthanakul, C. Kaplanis, N. Pawlowski, and M. Shanahan, "Feature control as intrinsic motivation for hierarchical reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3409–3418, Nov. 2019.

[6] C. Sun, W. Liu, and L. Dong, "Reinforcement learning with task decomposition for cooperative multiagent systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 5, pp. 2054–2065, May 2021.

[7] Z. Yang, K. Merrick, L. Jin, and H. A. Abbass, "Hierarchical deep reinforcement learning for continuous action control," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5174–5184, Nov. 2018.

[8] P. Dayan and G. E. Hinton, "Feudal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 5. San Mateo, CA, USA: Morgan Kaufmann, 1993, pp. 271–278.

[9] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.

[10] S. Pateria, B. Subagdja, and A. H. Tan, "Hierarchical reinforcement learning with integrated discovery of salient subgoals," in *Proc. 19th Int. Conf. Auto. Agents MultiAgent Syst. (AAMAS)*, Richland, SC, USA: International Foundation for Autonomous Agents and Multiagent Systems, 2020, pp. 1963–1965.

[11] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.

[12] M. Andrychowicz *et al.*, "Hindsight experience replay," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30. Red Hook, NY, USA: Curran Associates, 2017, pp. 5048–5058.

[13] C. Shi, R. Huang, and Z. Shi, "Automatic discovery of subgoals in reinforcement learning using unique-Dreiction value," in *Proc. 6th IEEE Int. Conf. Cognit. Informat.*, Aug. 2007, pp. 361–368.

[14] O. Şimşek and A. G. Barto, "Skill characterization based on betweenness," in *Proc. 21st Int. Conf. Neural Inf. Process. Syst. (NIPS)*. Red Hook, NY, USA: Curran Associates, 2008, pp. 1497–1504.

[15] S. Mannor, I. Menache, A. Hoze, and U. Klein, "Dynamic abstraction in reinforcement learning via clustering," in *Proc. 21st Int. Conf. Mach. Learn. (ICML)*, 2004, p. 71.

[16] I. Menache, S. Mannor, and N. Shimkin, "Q-cut—Dynamic discovery of sub-goals in reinforcement learning," in *Proc. 13th Eur. Conf. Mach. Learn. (ECML)*. Berlin, Germany: Springer, 2002, pp. 295–306.

[17] Ö. Şmşek, A. P. Wolfe, and A. G. Barto, "Identifying useful subgoals in reinforcement learning by local graph partitioning," in *Proc. 22nd Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 816–823.

[18] M. C. Machado, M. G. Bellemare, and M. Bowling, "A Laplacian framework for option discovery in reinforcement learning," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, vol. 70, 2017, pp. 2295–2304.

[19] A. Srinivas, R. Krishnamurthy, P. Kumar, and B. Ravindran, "Option discovery in hierarchical reinforcement learning using spatio-temporal clustering," 2016, *arXiv:1605.05359*. [Online]. Available: http://arxiv.org/abs/1605.05359

[20] S. Zhou, Z. Xu, and F. Liu, "Method for determining the optimal number of clusters based on agglomerative hierarchical clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 12, pp. 3007–3017, Dec. 2017.

[21] P. Yildirim and D. Birant, "K-linkage: A new agglomerative approach for hierarchical clustering," *Adv. Electr. Comput. Eng.*, vol. 17, no. 4, pp. 77–88, 2017.

[22] W. Bi and J. T. Kwok, "Mandatory leaf node prediction in hierarchical multilabel classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 12, pp. 2275–2287, Dec. 2014.

[23] J.-Y. Park and J.-H. Kim, "Incremental class learning for hierarchical classification," *IEEE Trans. Cybern.*, vol. 50, no. 1, pp. 178–189, Jan. 2020.

[24] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *Proc. 32nd Int. Conf. Mach. Learn.*, vol. 37. Lille, France: PMLR, Jul. 2015, pp. 1312–1320.

[25] Y. Duan, X. Chen, R. Houthooft, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *Proc. Mach. Learn. Res.*, vol. 48. New York, NY, USA: PMLR, Jun. 2016, pp. 1329–1338.

[26] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, nos. 1–2, pp. 181–211, 1999.

[27] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. 4th Int. Conf. Learn. Represent., (ICLR)*, San Juan, Puerto Rico, May 2016, pp. 1–14.

[28] P.-L. Bacon, "On the bottleneck concept for options discovery: Theoretical underpinnings and extension in continuous state spaces," M.S. thesis, School Comput. Sci., McGill Univ. Libraries, Montreal, QC, Canada, 2014. [Online]. Available: https://books.google.com.sg/books?id=40WSoAEACAAJ

[29] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst. (NIPS)*. Red Hook, NY, USA: Curran Associates, 2016, pp. 1479–1487.

[30] C. Moenning and N. A. Dodgson, "Fast marching farthest point sampling," Univ. Cambridge, Computer Lab., Cambridge, U.K., Tech. Rep. UCAM-CL-TR-562, Apr. 2003. [Online]. Available: https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-562.pdf

[31] N. Savinov, A. Dosovitskiy, and V. Koltun, "Semi-parametric topological memory for navigation," in *Proc. 6th Int. Conf. Learn. Represent., (ICLR)*, Vancouver, BC, Canada, May 2018, pp. 1–16.

[32] B. Eysenbach, R. R. Salakhutdinov, and S. Levine, "Search on the replay buffer: Bridging planning and reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 15246–15257.

Shubham Pateria received the B.Tech. degree in electronics and communications engineering from the National Institute of Technology (NIT), Durgapur, India, in 2013. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, Nanyang Technological University (NTU), Singapore.

He worked as a Senior Software Engineer with Samsung Research and Development Institute India-Bangalore (SRI-B), Bengaluru, India. His current research interests include reinforcement learning, representation learning for reinforcement learning, and self-supervised learning.

Budhitama Subagdja received the B.Sc. and M.Sc. degrees from the Faculty of Computer Science, Universitas Indonesia (UI), Depok, Indonesia, in 1995 and 2000, respectively, and the Ph.D. degree in information systems from the Department of Information Systems, University of Melbourne, Melbourne, VIC, Australia, in 2007.

He worked as a Research Assistant and a Lecturer with UI. He was a Senior Research Fellow with Nanyang Technological University (NTU), Singapore. He is currently a Research Scientist with the School of Information Systems, Singapore Management University (SMU), Singapore. His current research interests include biologically inspired cognitive architecture for intelligent agents, self-awareness in social robots, planning, reasoning, and reinforcement learning in autonomous agents and multiagent systems.

Ah-Hwee Tan (Senior Member, IEEE) received the B.Sc. (Hons.) and M.Sc. degrees in computer and information science from the National University of Singapore, Singapore, in 1989 and 1991, respectively, and the Ph.D. degree in cognitive and neural systems from Boston University, Boston, MA, USA, in 1994.

He is currently a Professor of computer science and an Associate Dean of research with the School of Computing and Information Systems, Singapore Management University (SMU), Singapore. His current research interests include cognitive and neural systems, brain-inspired intelligent agents, machine learning, knowledge discovery, and text mining.

Dr. Tan is an Associate Editor of the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.

Chai Quek (Senior Member, IEEE) received the B.Sc. and Ph.D. degrees from Heriot-Watt University, Edinburgh, U.K., in 1986 and 1990, respectively.

He is currently with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. He has authored or coauthored over 250 international conference and journal articles. His research interests include neurocognitive informatics, biomedical engineering, and computational finance.

Dr. Quek has been invited as a Program Committee Member and a Reviewer for several conferences and journals, including the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS (TNNLS) and TEvC.

Authorized licensed use limited to: Biruni Universitesi. Downloaded on November 13,2022 at 04:09:12 UTC from IEEE Xplore. Restrictions apply.